

Beyond the Syllabus

The Complexity Zoo, and Three 2024–2025 Breakthroughs in Algorithms
Week 2 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

- 1 Motivation
- 2 The Complexity Zoo
- 3 Recent Advances I: Matrix Multiplication After Strassen
- 4 Recent Advances II: Shortest Paths Beyond Dijkstra
- 5 Recent Advances III: Coloring Graphs Near-Optimally
- 6 Summary

Why look beyond the syllabus?

- This course teaches you **tools**: asymptotic analysis, greedy algorithms, divide and conquer, dynamic programming, network flow, NP-completeness.
- These tools are not museum pieces. Researchers are actively using them **right now** to settle questions that were open for 40–60 years.
- Today's supplement has two parts:
 - ① A tour of the **complexity zoo** – far more classes than just P and NP – with a worked example for each one, so “how hard is this, provably?” becomes a concrete question, not just vocabulary.
 - ② Three **2024–2025 breakthroughs**, each with a hand-checkable worked example, that broke a decades-old speed barrier using exactly the kind of reasoning ($T(n) = \dots$, recurrences, worst-case analysis) you are learning this semester.

The message

“Textbook algorithm” does not mean “final answer.” Dijkstra's algorithm is 70 years old and still generating Symposium-on-Foundations-of-Computer-Science best-paper awards.

What is a complexity class?

Definition (informal)

A **complexity class** is a set of computational problems that can be solved using a certain *amount* of a certain *resource* – as a function of input size n .

- The resource is not always **time**. It can be **space** (memory), **randomness**, or even the number of **quantifier alternations** in a logical formula – each gives a genuinely different classification of problems.
- P and NP (Week 12 preview) are the two most famous classes, but they are just two rooms in a much bigger zoo. Today we visit six more rooms: **L**, **NL**, **coNP**, **PSPACE**, **BPP/RP/ZPP**, and **#P**.
- Why bother? If a problem is provably outside P, you stop hunting for a fast exact algorithm and pivot – to approximation (Week 14), randomization (Week 15), or exponential-but-practical search.

P and EXP: the two time-based anchors

P (Polynomial time)

Decision problems solvable in $O(n^k)$ time, for some constant k .

EXP (Exponential time)

Decision problems solvable in $O(2^{n^k})$ time, for some constant k .

n	n^2 (P-ish)	n^5 (P-ish)	2^n (EXP-ish)	$n!$ (worse than EXP)
10	100	10^5	1,024	3.6×10^6
20	400	3.2×10^6	$\approx 10^6$	2.4×10^{18}
50	2,500	3.1×10^8	$\approx 10^{15}$	(astronomical)

- Shortest paths, sorting, matching, network flow – almost everything in Weeks 3–11 lives in P. A classical diagonalization argument proves **P** $\neq$ **EXP**: *something* in this zoo is provably hard, even though we can't always pin down *which familiar problem* it is.

Space matters too: L, NL, and PSPACE

L and NL (Logarithmic space)

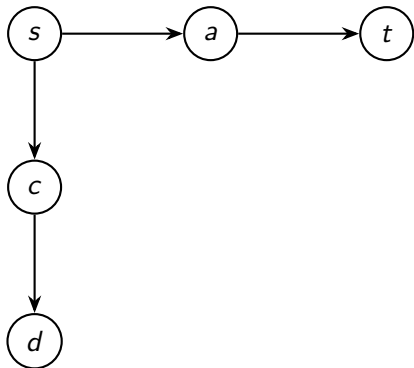
L: decision problems solvable using only $O(\log n)$ bits of *working memory* (the input itself is read-only and doesn't count). **NL**: the same, but the algorithm may guess (nondeterminism).

PSPACE (Polynomial space)

Decision problems solvable using $O(n^k)$ bits of working memory – with **no limit on time** at all.

- $O(\log n)$ memory is enough to store a handful of vertex labels or counters, but not an entire path or the whole input – a much tighter budget than “polynomial time.”
- Known: **L** $\subseteq$ **NL** $\subseteq$ **P** $\subseteq$ **PSPACE** $\subseteq$ **EXP**. Whether $L = NL$ is open – a space-world cousin of P vs. NP .

Worked example: PATH is NL-complete



Question: is there a directed path from s to t ?

- Path $s \rightarrow a \rightarrow t$ exists: **yes**.
- A logspace algorithm never stores the whole path – only the **current vertex** and a **step counter** ($O(\log n)$ bits each).
- *Nondeterministically* guess the next vertex at each step; accept if t is reached within n steps.

A genuine surprise

Immerman and Szelepcsényi (1987–88) proved **NL = coNL**: nondeterministic logspace is closed under complement – *despite* nondeterminism seeming “one-directional.” It took 20 years after NL was defined to discover this.

NP: efficiently *verifiable* problems

NP (Nondeterministic Polynomial time)

Decision problems where, *if the answer is “yes,”* there is a certificate that can be **checked** in polynomial time.

- Every problem in P is also in NP : if you can *solve* it fast, you can certainly *verify* a solution fast. So $P \subseteq NP$.
- The single biggest open question in computer science: **is $P = NP$?** Can everything easy to *check* also be made easy to *solve*?

Worked example: verifying a Subset-Sum certificate

Instance: $S = \{3, 34, 4, 12, 5, 2\}$, target $T = 9$. **Question:** does some subset of S sum to exactly T ?

- *Solving* this from scratch means searching (in the worst known case) over all $2^{|S|}$ subsets – exponential.
- But someone can hand you a **certificate**: the subset $\{4, 5\}$.

$$4 + 5 = 9 = T. \quad \checkmark$$

The point of NP

Checking the certificate took one addition and one comparison: $O(|S|)$ time, regardless of how hard it was to *find* $\{4, 5\}$ in the first place. That gap between “hard to find” and “easy to check” is exactly what NP captures.

NP-complete and NP-hard

- A problem is **NP-hard** if every problem in NP reduces to it in polynomial time – “at least as hard as anything in NP.”
- A problem is **NP-complete** if it is NP-hard *and* itself in NP – the “hardest problems inside NP.”
- Cook and Levin (1971) showed SAT (Boolean satisfiability) is NP-complete. Thousands of problems are now known to be NP-complete, including **Subset Sum** itself, the **Traveling Salesman Problem** (decision version), 3-COLORING, and VERTEX COVER – formalized in Week 12.

Why this matters in practice

Reduce a known NP-complete problem to *your* problem, and you have strong evidence it has **no** polynomial-time exact algorithm (unless $P = NP$) – your cue to switch strategies.

coNP: the mirror image of NP

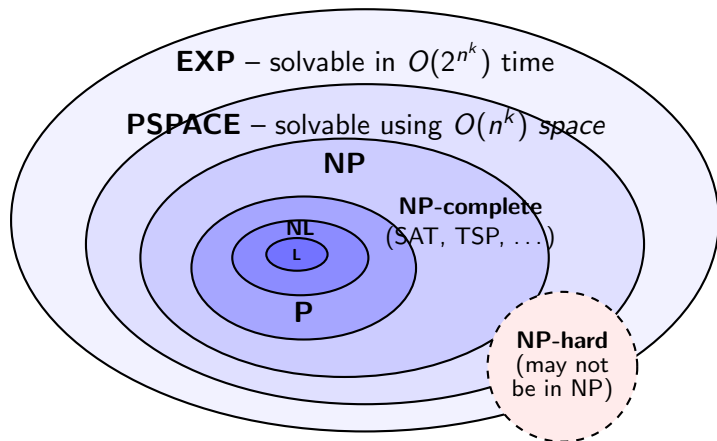
coNP

The class of problems whose “no” answers have polynomial-time-checkable certificates.

Formally: $L \in \text{coNP} \iff \bar{L} \in \text{NP}$.

- Example: TAUTOLOGY – “is this Boolean formula true under *every* assignment?” A “no” certificate is a single falsifying assignment, e.g. for $\varphi = (x \vee y) \wedge (\neg x \vee \neg y)$, the assignment $x=y=\text{true}$ falsifies it – checkable instantly.
- But a “yes” certificate would seem to require checking *all* 2^n assignments – no shortcut is known.
- Believed (not proven) that **NP** $\neq$ **coNP**: SAT’s “no” instances (unsatisfiability) are not believed to have short certificates either.

The big picture (time and space classes)



- Known for sure: $\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXP}$. Not shown to scale: **coNP** (NP's mirror, conjectured $\neq \mathbf{NP}$), **BPP/RP/ZPP** (randomized, next), **#P** (counting, later).
- **Not** known: whether any inclusion above is *strict* except $\mathbf{P} \neq \mathbf{EXP}$

Worked example: quantifiers and PSPACE

TQBF (True Quantified Boolean Formula) is the canonical **PSPACE-complete** problem: decide if a fully-quantified Boolean formula is true.

$$\varphi = \exists x \forall y (x \vee y)$$

- Try $x = \text{true}$: then $\forall y (\text{true} \vee y)$ is true for *every* y (true, since $\text{true} \vee \text{anything} = \text{true}$).
- So $x = \text{true}$ witnesses the outer $\exists$: φ is **true**. ✓

Why this is PSPACE, not just NP

Each quantifier can be evaluated by *reusing* the same working memory for the next one (recursion on formula structure), so only $O(n)$ space is needed – but potentially exponentially many assignments get explored over time. **Savitch's theorem** (1970) shows nondeterministic and deterministic polynomial-space classes coincide: $NPSPACE = PSPACE$.

Randomized classes: BPP, RP, and ZPP

BPP / RP / ZPP

Problems solvable in polynomial time by an algorithm that flips coins, with **two-sided error** (BPP), **one-sided error** (RP), or **zero error but random running time** ($ZPP = RP \cap coRP$).

- Known: $P \subseteq ZPP \subseteq RP \subseteq BPP \subseteq PSPACE$.
- Widely **conjectured** (not proven) that $BPP = P$: every efficient randomized algorithm can, in principle, be derandomized – a major research theme connecting complexity theory and pseudorandomness.
- You will meet this whole family concretely in **Week 15** (randomized algorithms): primality testing, randomized quicksort, min-cut.

Worked example: Freivalds' algorithm (an RP-style check)

Problem: given $n \times n$ matrices A, B, C , is $C = AB$? Computing AB directly costs $O(n^\omega)$ (next section!). **Freivalds (1977):** pick a random $r \in \{0, 1\}^n$ and just check $A(Br) \stackrel{?}{=} Cr$ – $O(n^2)$ time.

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}, \quad C = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix} (= AB), \quad r = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$Br = \begin{pmatrix} 11 \\ 15 \end{pmatrix}, \quad A(Br) = \begin{pmatrix} 1(11) + 2(15) \\ 3(11) + 4(15) \end{pmatrix} = \begin{pmatrix} 41 \\ 93 \end{pmatrix}, \quad Cr = \begin{pmatrix} 41 \\ 93 \end{pmatrix} \cdot \checkmark$$

Now suppose someone claims $C' = \begin{pmatrix} 19 & 22 \\ 43 & 51 \end{pmatrix}$ (one wrong entry): $C'r = \begin{pmatrix} 41 \\ 94 \end{pmatrix} \neq \begin{pmatrix} 41 \\ 93 \end{pmatrix}$ – **caught**.

The guarantee

If $AB \neq C$, a random r catches it with probability $\geq \frac{1}{2}$. Repeat k times independently: error probability $\leq 2^{-k}$.

Counting complexity: #P and the permanent

#P

Instead of “does a solution exist?” (NP), #P asks “**how many solutions are there?**” – e.g., #SAT: how many satisfying assignments does φ have?

For a 2×2 matrix $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, compare two almost-identical-looking formulas:

$$\det(M) = ad - bc \quad \text{vs.} \quad \text{perm}(M) = ad + bc.$$

With $M = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$: $\det(M) = 6 - 1 = 5$, but $\text{perm}(M) = 6 + 1 = 7$.

A striking contrast

$\det(M)$ is computable in polynomial time (Gaussian elimination, or the same machinery as matrix multiplication, next section). Valiant (1979) proved computing $\text{perm}(M)$ exactly is **#P-complete** – provably as hard as counting *all* satisfying assignments of a SAT formula – even though, for a 0/1 matrix, the permanent is just “the number of perfect matchings” in a bipartite graph (Weeks 10–11). Two formulas that differ only by a sign; wildly different difficulty.

P vs. NP: the \$1,000,000 question

- P vs. NP is one of the seven **Clay Millennium Prize Problems** (\$1,000,000 for a correct proof either way). Only one has been solved so far (the Poincaré conjecture, by Grigori Perelman, 2003).
- Almost every theorist believes $P \neq NP$ – but belief is not proof, and the problem has resisted every attack for over 50 years.
- **Why you should care as a 2nd-year student:** every time you face a new problem in industry or research, “is this secretly NP-hard?” is one of the first questions worth asking, *before* you spend a week trying to write a fast exact algorithm.

Looking ahead

Week 12 makes all of this precise: formal reductions, Cook–Levin, and a toolkit for proving your own problems NP-hard.

Recap: Strassen's trick (1969)

- You saw last week that Karatsuba traded one multiplication for extra additions to beat the $\Theta(n^2)$ grade-school bound.
- Volker Strassen applied the *same philosophy* to **matrix multiplication**: multiplying two 2×2 block matrices needs only **7** multiplications (of the blocks), not the obvious **8**.
- Recursively applied to $n \times n$ matrices:

$$T(n) = 7 T(n/2) + O(n^2) \implies T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807}),$$

beating the schoolbook $\Theta(n^3)$ – the first improvement in recorded history.

Worked example: Strassen's seven products

Let $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. Direct computation: $AB = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$ (8 multiplications). **Strassen's seven products:**

$$p_1 = a_{11}(b_{12} - b_{22}) = 1(6 - 8) = -2,$$

$$p_2 = (a_{11} + a_{12})b_{22} = 3(8) = 24,$$

$$p_3 = (a_{21} + a_{22})b_{11} = 7(5) = 35,$$

$$p_4 = a_{22}(b_{21} - b_{11}) = 4(2) = 8,$$

$$p_5 = (a_{11} + a_{22})(b_{11} + b_{22}) = 5(13) = 65,$$

$$p_6 = (a_{12} - a_{22})(b_{21} + b_{22}) = (-2)(15) = -30,$$

$$p_7 = (a_{11} - a_{21})(b_{11} + b_{12}) = (-2)(11) = -22.$$

Combine (only additions from here):

$$c_{11} = p_5 + p_4 - p_2 + p_6 = 19, \quad c_{12} = p_1 + p_2 = 22, \quad c_{21} = p_3 + p_4 = 43, \quad c_{22} = p_5 + p_1 - p_3 - p_7 = 50.$$

Matches AB exactly ✓ – using **7**, not 8, multiplications.

The exponent ω : a 50-year race

Define ω as the smallest exponent such that $n \times n$ matrices can be multiplied in $O(n^{\omega+\varepsilon})$ time for every $\varepsilon > 0$.

Year	Result	Bound on ω
1969	Strassen	2.807...
1990	Coppersmith–Winograd	2.376
2014	Le Gall	2.3728639
2020–21	Alman–Williams	2.3728596
2022	Duan–Wu–Zhou	2.3719
2024	Williams–Xu–Xu–Zhou	2.371339

- Nobody knows the true value of ω ; many suspect $\omega = 2$ is achievable, but every attempt to prove it has stalled.
- These are all **asymptotic** (large- n) results with enormous constants – not (yet) practical for the matrix sizes you'd use in a homework assignment.

A different question: the small-case puzzle

- Separately from the asymptotic race: **exactly how many scalar multiplications are needed to multiply two 4×4 matrices?**
- Applying Strassen recursively to a 4×4 matrix (as two levels of 2×2 blocks) uses $7 \times 7 = \mathbf{49}$ multiplications.
- For over 50 years, **49** was the best known scheme for 4×4 matrices – until 2022.

Why anyone cares about “49 vs. 47”

Small building blocks get used *recursively*. Shaving multiplications off the base case improves the constant in the asymptotic bound every level of recursion – exactly like the jump from 4 to 3 recursive calls did for Karatsuba.

AI-assisted discovery joins the search

Year	Who / what	Result for 4×4
1969–2021	Strassen (recursive)	49 multiplications
2022	DeepMind's AlphaTensor	47 multiplications (mod-2 arithmetic)
2024	Kauers & Moosbauer (JKU Linz)	48 multiplications (standard arithmetic)
2025	DeepMind's AlphaEvolve	48 multiplications (complex-valued)

- AlphaTensor (Nature, 2022) reframed matrix multiplication as a single-player game and used reinforcement learning to search for low-multiplication schemes – the first improvement on Strassen's 4×4 case in 55 years.
- AlphaEvolve (2025) is a large-language-model-driven *evolutionary code search*: it proposes, tests, and refines algorithm implementations directly.

The lesson

A problem being “settled since 1969” didn't mean it was actually settled – it meant nobody had looked hard enough, with the right tools.

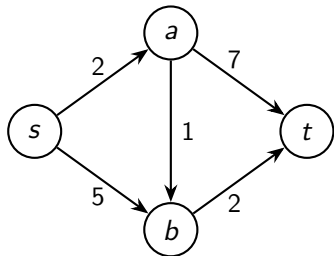
Dijkstra's algorithm (1956) and the sorting barrier

- **Single-source shortest paths:** given a graph with n vertices, m edges, and a source s , find the shortest distance from s to every other vertex.
- Dijkstra's algorithm: repeatedly pick the *closest unvisited* vertex, “finalize” its distance, and relax its outgoing edges.
- In 1984, Fredman and Tarjan's **Fibonacci heap** made this run in $O(m + n \log n)$ time – matching a fundamental lower bound: any algorithm that finalizes vertices *in sorted order of distance* cannot beat the time it takes to **sort** n numbers.

The “sorting barrier”

For 40 years, this looked like the end of the road for general (real-weighted) graphs – *unless* you find a way to avoid sorting altogether.

Worked example: Dijkstra finalizes in sorted order



Step	Finalize	Relax	New tentative dist.
1	$s(0)$	$s \rightarrow a, s \rightarrow b$	$a=2, b=5$
2	$a(2)$	$a \rightarrow b, a \rightarrow t$	$b=3, t=9$
3	$b(3)$	$b \rightarrow t$	$t=5$
4	$t(5)$	–	–

This *is* the sorting barrier

Finalization order s, a, b, t is exactly increasing distance order $0 < 2 < 3 < 5$. Dijkstra's algorithm is, structurally, sorting the vertices by distance as it goes – so it cannot run faster than sorting allows.

2024: Dijkstra is “universally optimal”

- Haeupler, Rozhoň, Tětek, Hladík, and Tarjan (FOCS 2024 best paper) asked a different question: not “what is the worst case over *all* graphs,” but **“is there one algorithm that is the best possible on every individual graph structure?”**
- They designed a new heap data structure and proved: a variant of Dijkstra’s algorithm is **universally optimal** – for every fixed graph *shape*, no algorithm can beat it (for worst-case edge weights on that shape).
- A beautiful, 70-years-later vindication of Dijkstra’s original 20-minute, pencil-free design.

2025: breaking the sorting barrier

- Just one year later, Duan, Mao, and collaborators (STOC 2025 best paper) proved the opposite-sounding result: an algorithm that **beats** Dijkstra's $O(m + n \log n)$ *worst-case* bound, on general directed, real-weighted graphs.
- Key idea: don't finalize vertices strictly by increasing distance.
 - Group nearby frontier vertices into **clusters**; explore one representative per cluster instead of scanning the whole frontier.
 - Use short, bounded bursts of the (usually slow) **Bellman–Ford** algorithm to find “influential” hub vertices worth exploring next.
- Because vertices are no longer finalized in sorted order (unlike our s, a, b, t example above), the sorting lower bound simply **does not apply**.

Reconciling the two results

2024 (Haeupler et al.)	2025 (Duan, Mao et al.)
"Best possible <i>per graph shape</i> ," among algorithms that finalize vertices by increasing distance	"Faster <i>in the worst case over all graphs</i> ," by abandoning distance-order finalization entirely

The teaching moment

Both statements are true. "Optimal" always means *optimal with respect to a model and a metric*. Changing the model (what the algorithm is allowed to do) or the metric (worst-case-over-graphs vs. per-graph-shape) can change the answer – a theme you will see again with approximation ratios in Week 14.

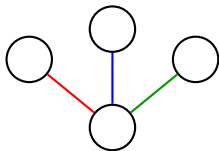
The problem: edge coloring

- **Motivating story:** at an airport, model runways/taxiways/gates as vertices and each aircraft's planned route as a path connecting them. Color each edge (route segment) so that no two edges sharing a vertex share a color – colors represent non-conflicting time slots.
- **Vizing's theorem** (1964): every graph can be edge-colored with at most $\Delta + 1$ colors, where Δ is the maximum vertex degree. Knowing *how many* colors suffice is easy.
- **Actually finding** such a coloring quickly is a different, much harder story.

Worked example: when is $\Delta + 1$ actually necessary?

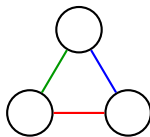
Star $K_{1,3}$: $\Delta = 3$

3 colors suffice ($= \Delta$): **Class 1**



Triangle K_3 : $\Delta = 2$

3 colors needed ($= \Delta + 1$): **Class 2**



Why the triangle needs an extra color

Every pair of the triangle's 3 edges shares an endpoint, so *all three* must get different colors – $\Delta = 2$ colors are not enough. Vizing's theorem's “+1” is sometimes unavoidable, and telling class 1 from class 2 graphs is itself NP-hard in general.

A 40-year plateau

- Vizing's own algorithm runs in $O(mn)$ time (recoloring a chain of edges can cascade across the whole graph).
- 1980s improvements brought this down to $O(m\sqrt{n})$ – and then progress **stalled for roughly 40 years**.
- “The research stopped for a very long time ... many people believed that there's no better way,” as one researcher put it.

2024–2025: near-linear time, finally

- 2024: two independent groups broke the 40-year plateau within days of each other: Assadi's $O(n^2)$ -time algorithm, and a separate $O(m\sqrt[3]{n})$ -time algorithm, later refined to $O(m\sqrt[4]{n})$.
- 2025: Assadi, Behnezhad, Costa, and collaborators combined forces and went much further – a **near-linear** $O(m \log m)$ -time algorithm, presented at STOC 2025.
- Since simply *reading* the graph takes $\Omega(m)$ time, $O(m \log m)$ is within a single logarithmic factor of the **theoretical minimum possible**.

The key new idea: “priming”

Rather than trying to color the *last, hardest* edge faster (the old approach), use randomization to color *almost all* edges at once, leaving only a small, structured set of edges – which can then be colored quickly and nearly simultaneously.

Why these three stories matter

- Every one of these breakthroughs attacks a **classical, textbook** problem: matrix multiplication, shortest paths, edge coloring.
- Every one of these barriers stood for **decades** (40–55 years) and was widely believed to be permanent.
- Every one of these breakthroughs came from a genuinely **new idea** about combining old techniques (clustering, bounded Bellman–Ford bursts, randomized “priming”), not from more computing power alone – though AI-assisted search (AlphaTensor, AlphaEvolve) is now a serious new tool in the theorist’s toolbox.

Summary

- 1 **The complexity zoo** – L, NL, P, NP, coNP, PSPACE, EXP (time/space), plus BPP/RP/ZPP (randomness) and $\#P$ (counting) – gives you a vocabulary for asking “how hard, provably, is this problem, and with respect to what resource?”
- 2 **Matrix multiplication:** Strassen’s 7-product trick generalizes to an asymptotic exponent ω that keeps inching down ($2.807 \rightarrow 2.371$), and AI-assisted search (AlphaTensor, AlphaEvolve) has now improved even the small, 55-year-old 4×4 case.
- 3 **Shortest paths:** Dijkstra’s algorithm was proved *universally optimal* in 2024 – and its worst-case bound was still *beaten* in 2025, by an algorithm that refuses to sort. Both are correct; they optimize different things.
- 4 **Edge coloring:** a 40-year-stalled problem fell to near-linear time in 2024–2025 via a new “priming” idea.

Looking ahead

The tools behind every one of these results – recurrences, worst-case analysis, reductions, randomization – are exactly what the rest of this course builds up, week by week.